Appointment API

Frontend Integration Guide

Version 1.0 | Production-ready reference for web & mobile clients

1. Overview

This guide describes how frontend applications (web or mobile) should integrate with the Appointment API. It covers authentication, endpoints, request/response contracts, status lifecycle, permissions, filtering, pagination, error handling, and the notifications the client can expect.

All endpoints require authentication. Role-based access control is enforced server-side: patients, doctors, and staff have different capabilities.

2. Authentication

Every request must include a valid authentication credential (typically a Bearer JWT or session cookie, depending on your project's auth setup).

Example header:

```
Authorization: Bearer <access_token>
```

Unauthenticated requests receive HTTP 401. Users without a patient or doctor profile (and who are not staff) receive HTTP 403 on appointment endpoints.

3. Base Conventions

- Base path: `/api/appointments/`
- Content-Type: `application/json`
- Dates: `YYYY-MM-DD` (e.g. 2026-09-15)
- Times: `HH:MM:SS` or `HH:MM` (24-hour, local clinic time)
- All timestamps returned by the API are ISO 8601 datetime strings.

4. Appointment Status Lifecycle

Appointments follow a strict state machine. The frontend should use the can_* flags returned in the response to decide which action buttons to show.

Status	Meaning	Can transition to	Who can act
pending	Created, awaiting doctor confirmation	confirmed, cancelled	Patient/Doctor/Staff (cancel); Doctor/Staff (confirm)
confirmed	Doctor accepted the appointment	completed, no_show, cancelled	Doctor/Staff (complete/no-show); Patient/Doctor/Staff (cancel)
completed	Visit finished successfully	— (terminal)	—
cancelled	Cancelled by patient, doctor or staff	— (terminal)	—
no_show	Patient did not attend	— (terminal)	—

Reschedule note: Rescheduling a confirmed appointment resets its status back to pending (and clears confirmed_at).

5. Endpoints

5.1 List Appointments

`GET /api/appointments/`

Auth: Required (Patient sees own, Doctor sees own, Staff sees all)

Query parameters:

Parameter	Type	Description
status	string	Comma-separated list, e.g. pending,confirmed
date	YYYY-MM-DD	Exact appointment date
from	YYYY-MM-DD	Appointments on or after this date
to	YYYY-MM-DD	Appointments on or before this date
upcoming	boolean	true / 1 / yes — only future non-terminal appointments
page	integer	Page number (default 1)
page_size	integer	Items per page (default 20, max 100)

Response (200): Paginated list. Each item is a full Appointment object (see section 6).

5.2 Create Appointment

`POST /api/appointments/`

Auth: Patient only

Request body:

```
{ "doctor": 12, "appointment_date": "2026-09-15", "appointment_time": "10:30",
"reason_for_visit": "Annual checkup" }
```

Notes:

- duration_minutes is set automatically from the doctor's consultation duration.
- status is always set to pending on creation.
- Server validates: not in the past, fits doctor availability, no overlap with existing active appointments.
- On success the server sends notifications to both patient and doctor.

Success: 201 Created — full Appointment object.

Conflict: 409 — slot taken by a concurrent booking.

5.3 Retrieve Appointment

`GET /api/appointments/{id}/`

Auth: Patient of the appointment, assigned doctor, or staff.

5.4 Cancel Appointment

`POST /api/appointments/{id}/cancel/`

Auth: Patient, assigned doctor, or staff.

Request body (optional):

```
{ "cancellation_reason": "Unable to attend" }
```

Only allowed when status is pending or confirmed. Both parties are notified.

5.5 Confirm Appointment

`POST /api/appointments/{id}/confirm/`

Auth: Assigned doctor or staff. Only pending → confirmed. Patient is notified.

5.6 Complete Appointment

`POST /api/appointments/{id}/complete/`

Auth: Assigned doctor or staff. Only confirmed → completed. Patient is notified.

5.7 Mark No-Show

`POST /api/appointments/{id}/no-show/`

Auth: Assigned doctor or staff. Only confirmed → no_show. Patient may be notified.

5.8 Reschedule Appointment

`POST /api/appointments/{id}/reschedule/`

Auth: Patient of the appointment or staff.

Request body:

```
{ "appointment_date": "2026-09-20", "appointment_time": "14:00", "reason_for_visit": "Optional updated reason" }
```

Only pending or confirmed appointments can be rescheduled. If the appointment was confirmed, status is reset to pending. Both parties are notified with old and new times.

6. Appointment Response Object

All successful read and mutation endpoints return (or include) an object with the following shape:

Field	Type	Notes
id	integer	Primary key
patient	integer	Patient ID
patient_name	string	Read-only display name
doctor	integer	Doctor ID
doctor_name	string	Read-only display name
doctor_specialty	string|null	Specialty name
appointment_date	date	YYYY-MM-DD
appointment_time	time	Start time
end_time	time	Computed end time
duration_minutes	integer	Set from doctor
status	string	pending | confirmed | completed | cancelled | no_show
status_display	string	Human-readable status
reason_for_visit	string	Patient-provided reason
confirmed_at	datetime|null	When confirmed
completed_at	datetime|null	When completed
cancelled_at	datetime|null	When cancelled
cancelled_by	integer|null	User ID who cancelled

cancelled_by_name	string|null	Display name
cancellation_reason	string	Optional reason
check_in	object|null	Check-in details if present
is_upcoming	boolean	Computed convenience flag
can_cancel	boolean	UI helper – current user may cancel
can_confirm	boolean	UI helper – current user may confirm
can_complete	boolean	UI helper – current user may complete
can_mark_no_show	boolean	UI helper – current user may mark no-show
created_at	datetime	Creation timestamp
updated_at	datetime	Last update timestamp

Use the can_* boolean flags to show or hide action buttons. Do not re-implement permission logic on the client.

7. Permission Matrix

Action	Patient	Doctor	Staff
List own	Yes	Yes	All
Create	Yes	No	No*
Retrieve	Own only	Own only	Any
Cancel	Own	Own	Any
Confirm	No	Own	Any
Complete	No	Own	Any
Mark no-show	No	Own	Any
Reschedule	Own	No	Any

** Staff creation of appointments on behalf of patients is not exposed in the current API.*

8. Notifications

The backend pushes in-app (and optionally push/email) notifications on key events. The frontend should listen for notification events or poll the notifications endpoint. Each notification includes data.appointment_id and data.event.

Event	Recipients	Typical title / intent
created	Patient + Doctor	Appointment Scheduled / New Appointment Request
confirmed	Patient	Appointment Confirmed
cancelled	Patient + Doctor	Appointment Cancelled (includes actor)
rescheduled	Patient + Doctor	Appointment Rescheduled (old → new time)
completed	Patient	Appointment Completed
no_show	Patient	Missed Appointment

9. Error Handling

The API uses standard HTTP status codes. Validation errors return field-level messages.

Status	Meaning / Frontend action
400	Validation or illegal state transition. Show field errors or the detail message.
401	Not authenticated. Redirect to login / refresh token.
403	Authenticated but not allowed. Hide the action or show an access-denied message.
404	Appointment not found (or not visible to the user).
409	Slot conflict (create/reschedule). Prompt user to pick another time.

Example validation error (400):

```
{ "appointment_time": ["The selected time overlaps with another appointment."] }
```

10. Recommended Frontend Flows

Patient – Book an appointment

1. Select doctor (from a separate doctors API).
2. Select date & time that fits the doctor's availability.
3. Enter reason for visit.
4. POST /api/appointments/ → handle 201 or 409.
5. Show success state and the returned appointment (status = pending).

Doctor – Manage incoming requests

1. GET /api/appointments/?status=pending&upcoming=true
2. For each item, if can_confirm is true show a Confirm button → POST …/confirm/
3. After the visit, use can_complete / can_mark_no_show to finish the appointment.

Patient – Reschedule or cancel

- Only show Reschedule / Cancel when can_cancel is true (or status is pending/confirmed).
- On reschedule, capture the new date/time and call POST …/reschedule/. Expect status to become pending again if it was previously confirmed.

11. Frontend Best Practices

- Always trust the can_* flags from the API rather than hard-coding role logic.
- After any mutation, either use the returned appointment object or re-fetch the detail endpoint so the UI stays consistent.
- Handle 409 (conflict) gracefully on create and reschedule by prompting the user to choose another slot.
- Use the upcoming=true filter for "My upcoming appointments" lists.
- Display status_display to users; keep the raw status value for logic.
- Listen for real-time notifications (or poll) so the UI updates when the other party confirms, cancels, or reschedules.
- Never send duration_minutes or status on create — the server controls those fields.

Document generated for the Appointment API. For backend changes or additional endpoints, update this guide in lockstep.